

AWS Re-Architecting of Web Application

PaaS / SaaS Architecture on AWS Cloud

Author: Abdullah Mudassar | Role: DevOps Associate | AWS SAA-C03 Certified

Project Overview

Re-architected a Java web application from a self-managed stack to a fully managed, scalable AWS infrastructure using PaaS and SaaS services. The objective was to eliminate manual server management and move to a production-grade, cloud-native architecture.

Architecture Overview

Layer	Service
Application Tier	AWS Elastic Beanstalk (Tomcat 10 / Corretto 21)
Load Balancing	Application Load Balancer (ALB)
Auto Scaling	Elastic Beanstalk Auto Scaling Group
Database	Amazon RDS — MySQL 8.0
Caching	Amazon ElastiCache — Memcached 1.6
Message Broker	Amazon MQ — RabbitMQ
CDN	Amazon CloudFront
Artifact Storage	Amazon S3
SSL / TLS	AWS Certificate Manager (ACM)
DNS	GoDaddy CNAME + Route 53

Implementation Steps

1. Security Groups

- **Backend SG (vprofile-rearch-backend-sg)** — protects RDS, ElastiCache, and Amazon MQ
- **Inbound rule** — accepts traffic only from the Beanstalk security group — never exposed to the public internet

2. Key Pairs

→ **vprofile-rearch-key** — RSA key pair generated for EC2 SSH access during database initialization

3. Amazon RDS — MySQL 8.0

- **Parameter Group** — vprofile-rds-rearch-paragrp (MySQL 8.0 family)
- **Subnet Group** — vprofile-rds-rearch-subgrp (dedicated)
- **Instance** — vprofile-rds-rearch on db.t4g.micro in us-east-1a

4. Database Initialization

- **Step 1** — Launched temporary EC2 with MySQL client installed
- **Step 2** — Connected to RDS endpoint and imported schema from db_backup.sql (cloned from application repository)
- **Step 3** — Verified tables — role, user, user_role — created successfully
- **Step 4** — EC2 instance terminated immediately after initialization

5. Amazon ElastiCache — Memcached

- **Parameter Group** — vprofile-rearch-paragrp (Memcached 1.6 family)
- **Subnet Group** — vprofile-rearch-ca
- **Cluster** — vprofile-rearch-cache on cache.t3.micro

6. Amazon MQ — RabbitMQ

- **Broker** — vprofile-rearch-mq, single-instance on mq.t3.micro
- **Config** — Credentials configured in application.properties

7. IAM Roles

- **EC2 Instance Profile** — VPROFILE-REARCH-BEANROLE with required Beanstalk permissions
- **Service Role** — aws-elasticbeanstalk-service-role

8. Elastic Beanstalk Environment

- **Platform** — Tomcat 10 with Corretto 21 on 64-bit Amazon Linux 2023
- **Environment** — Load balanced — min 2, max 4 instances (t2.micro)
- **VPC** — Multi-AZ subnets configured

9. Application Configuration

Updated application.properties with:

- RDS JDBC endpoint and credentials
- ElastiCache Memcached host and port
- Amazon MQ RabbitMQ address and credentials

10. Build & Deploy

- **Build** — WAR artifact built using Maven (mvn install) — build completed in 19 seconds
- **Deploy** — vprofile-v2.war uploaded via Elastic Beanstalk Upload and Deploy
- **Policy** — Rolling — batched updates with no full downtime
- **Result** — Environment health OK, all 40 events successful

11. HTTPS Configuration

- **ALB Listener** — Port 443 with ACM SSL certificate (*.achaudhry.online)
- **TLS Policy** — ELBSecurityPolicy-TLS13-1-2-2021-06
- **Backend SG** — Updated to allow Beanstalk SG traffic on MySQL port 3306

12. DNS Configuration

- **GoDaddy CNAME** — vproarchabdullah.achaudhry.online → Beanstalk domain

13. Amazon CloudFront

- **Origin** — Elastic Beanstalk environment
- **Custom Domain** — vproabdullah.achaudhry.online with ACM TLS certificate
- **DNS** — GoDaddy CNAME pointing to CloudFront distribution domain

Verification

Service	Verification Method
Amazon RDS	Logged in via application at custom domain — user data loaded successfully
ElastiCache	First request: Data is From DB → Second request: Data is From Cache
Amazon MQ	Broker running, credentials validated via application properties

HTTPS

Connection secure on custom domain via ACM certificate

Deployment Policy Reference

Elastic Beanstalk deployment policies define how new application versions are released. This project used the Rolling policy.

Policy	Downtime	Cost	Risk	Used Here
All at once	Yes	Low	High	
Rolling	No	Low	Medium	✓
Rolling + extra batch	No	Medium	Low	
Immutable	No	High	Very Low	
Traffic splitting	No	Medium	Lowest	

Tech Stack

- Java (Spring MVC) — application framework
- Maven — build tool, WAR packaging
- AWS Elastic Beanstalk — Tomcat 10 / Corretto 21
- Amazon RDS — MySQL 8.0
- Amazon ElastiCache — Memcached 1.6
- Amazon MQ — RabbitMQ
- Amazon CloudFront — CDN
- Amazon S3 — artifact storage
- AWS Certificate Manager — SSL/TLS
- GoDaddy — domain and DNS management

Author

Abdullah Mudassar

DevOps Associate | AWS Certified Solutions Architect (SAA-C03) | Azure Fundamentals (AZ-900)

GitHub: github.com/AbdullahMudassar

LinkedIn: linkedin.com/in/abdullahmudassar